OPERATING SYSTEMS

UNIT-3

PRAVIN S. GAME, PH.D.
ASSOCIATE PROFESSOR
DEPARTMENT OF COMPUTER ENGINEERING
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

- **Course Objectives:**

1. To introduce the Operating system and process, threads and CPU scheduling
2. To explore inter process communication mechanisms, to introduce the critical-section problem and solutions.
3. To explore various techniques of allocating memory to processes and explain the concepts of demand paging, page-replacement algorithms.
4. To describe the details of implementing local file systems and directory structures

- **Course Outcomes:**

After learning the course, the students should be able to:

1. Compare various process scheduling algorithms for a given snapshot of the system.
2. Analyze IPC mechanisms and solutions of process synchronization for critical section problems.
3. Analyze the performance of memory management algorithms for a given problem.
4. Apply disk scheduling policies for a given I/O request sequence with file management concepts

PRAVIN S. GAME

UNIT - 3

- **Memory Management**

Introduction, Contiguous and non-contiguous, Swapping,

Memory Allocation Strategies, Paging, Segmentation,

Virtual Memory:

Background, Demand paging, Page Replacement Policies.

PRAVIN S. GAME

INTRODUCTION

- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of addresses + read requests, or address + data and write requests
- Register access takes one CPU clock (or less)
- Main memory can take many cycles
- **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation

INTRO…

➢ **Memory organization**

- Memory can be organized in different ways
  - One process uses entire memory space
  - Each process gets its own partition in memory
    - Dynamically or statically allocated
- Trend: Application memory requirements tend to increase over time to fill main memory capacities

Operating System	Release Date	Minimum Memory Requirement	Recommended Memory
Windows 1.0	November 1985	256KB	
Windows 2.03	November 1987	320KB	
Windows 3.0	March 1990	896KB	1MB
Windows 3.1	April 1992	2.6MB	4MB
Windows 95	August 1995	8MB	16MB
Windows NT 4.0	August 1996	32MB	96MB
Windows 98	June 1998	24MB	64MB
Windows ME	September 2000	32MB	128MB
Windows 2000 Professional	February 2000	64MB	128MB
Windows XP Home	October 2001	64MB	128MB
Windows XP Professional	October 2001	128MB	256MB
Windows 10	July 2015	2GB	
Windows 11	October 2021	4GB	

PRAVIN S. GAME

- Uniprogramming system – Two parts
  - OS
  - User program
- Multiprogramming system
  - Many user programs
- Strategies for obtaining optimal memory performance
  - Performed by memory manager
    - Which process will stay in memory?
    - How much memory will each process have access to?
    - Where in memory will each process go?

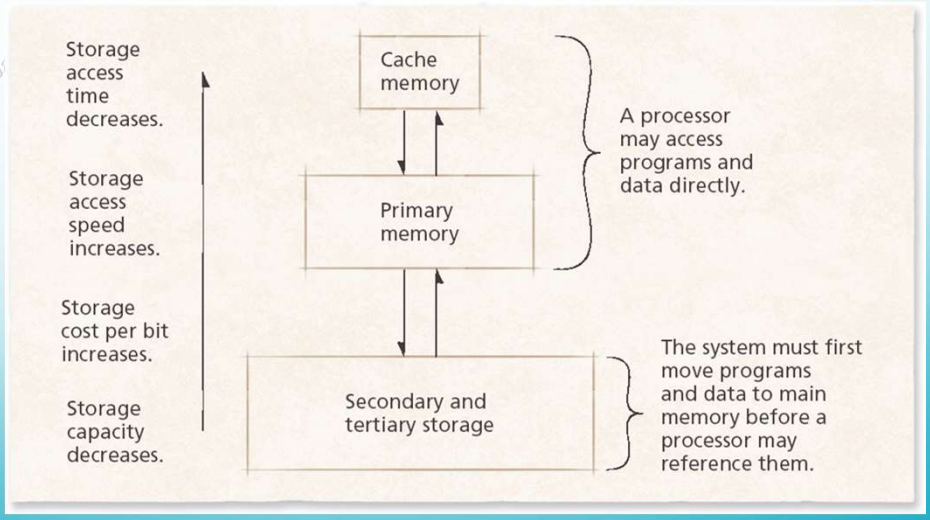
PRAVIN S. GAME

INTRO

➢Memory Hierarchy

- Main memory
  - Should store currently needed program instructions and data only
- Secondary storage
  - Stores data and programs that are not actively needed
- Cache memory
  - Extremely high speed
  - Usually located on processor itself
  - Most-commonly-used data copied to cache for faster access
  - Small amount of cache still effective for boosting performance
    - Due to temporal locality

PRAVIN S. GAME

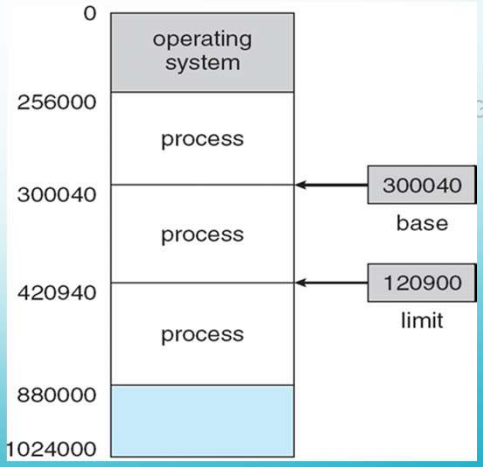


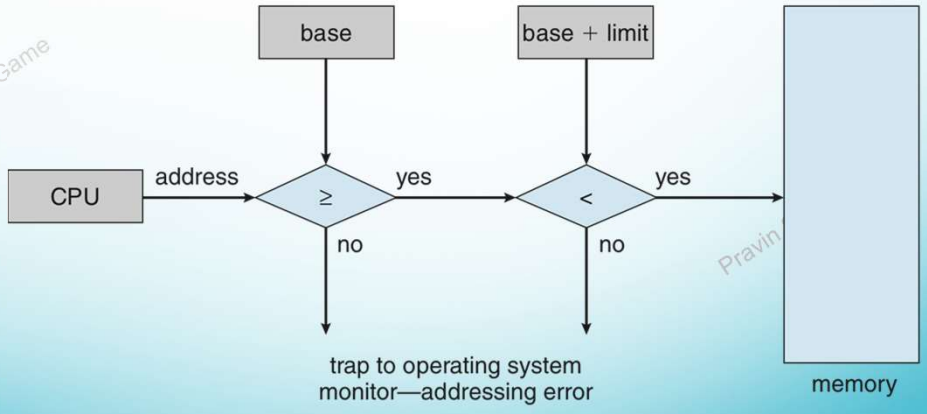
PRAVIN S. GAME

- Ways of organizing programs in memory
  - Contiguous allocation
    - Program must exist as a single block of contiguous addresses
    - Sometimes it is impossible to find a large enough block
    - Low overhead
  - Noncontiguous allocation
    - Program divided into chunks called segments
    - Each segment can be placed in different part of memory
    - Easier to find “holes” in which a segment will fit
    - Increased number of processes that can exist simultaneously in memory offsets the overhead incurred by this technique

BASIC HARDWARE

- Base and limit registers





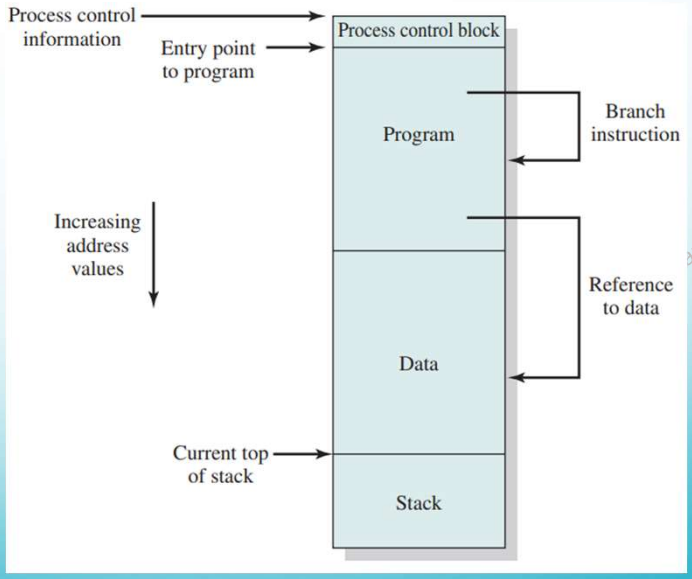
Hardware Address Protection with Base and Limit Registers

MEMORY MANAGEMENT REQUIREMENTS

- Relocation
- Protection
- Sharing
- Logical organization
- Physical organization

- Relocation
  - Available memory is shared between multiple processes
  - To maximise processor utilization active processes will be swapped in and out of memory
  - If a process is swapped to disk, it is difficult to place it at same memory location
  - So, need to relocate

PRAVIN S. GAME



Addressing Requirements for a Process

PRAVIN S. GAME

- Protection
  - programs in other processes should not be able to reference memory locations in a process.
  - satisfaction of the relocation requirement increases the difficulty of satisfying the protection requirement.
  - the memory protection requirement must be satisfied by the processor (hardware) rather than the operating system (software).

- Sharing
  - Any protection mechanism must have the flexibility to allow several processes to access the same portion of main memory.
  - controlled access to shared areas of memory without compromising essential protection

- Logical Organization
  - Main awa Secondary memory in a computer system is organized as a linear or one-dimensional address space.
  - Consist of a sequence of bytes or words.
  - It mirrors the machine hardware but not the programs
  - Programs have modules – some modify data, some do not
  - OS and H/w together can deal with modules : Advantageous

1. Modules can be written and compiled independently
2. Different degrees of protection can be given to different modules.
3. Modules can be shared among processes

Segmentation provides this facility.

- Physical Organization
  - Main Memory and Secondary memory
  - Organization of the flow of information between main and secondary memory is a major system concern
  - Can it be given to programmer?
  - The task of moving information between the two levels of memory should be a system responsibility

➢ Memory Management Strategies

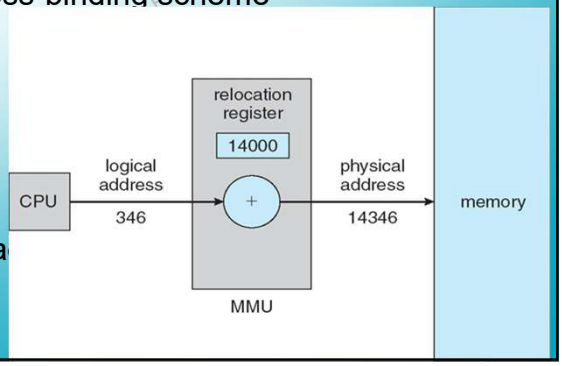
- Fetch strategies
  - Demand or anticipatory
  - Decides which piece of data to load next
- Placement strategies
  - Decides where in main memory to place incoming data
- Replacement strategies
  - Decides which data to remove from main memory to make more space

PRAVIN S. GAME

Logical vs. Physical Address Space

- **Logical address** – generated by the CPU; also referred to as **virtual address**
- **Physical address** – address seen by the memory unit; loaded into memory address register

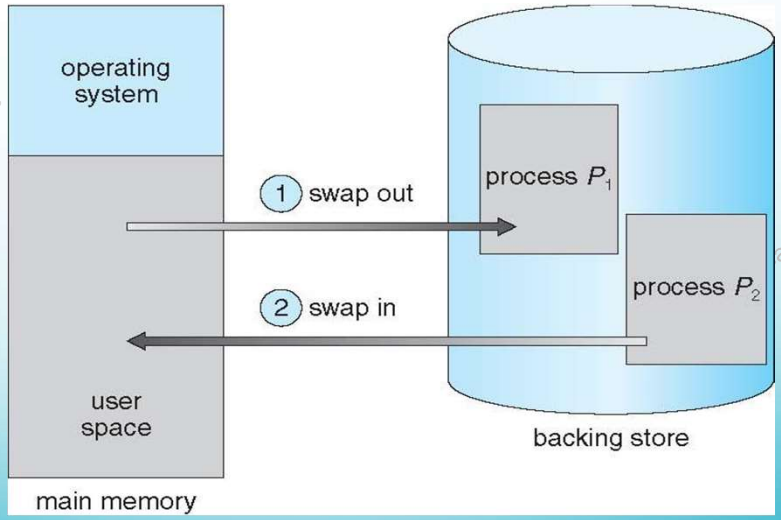
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- Logical and Physical address space
- Virtual to Physical address mapping at run-time is done by MMU
- Base register is called as relocation reg.
- User program never sees real physical a



PRAVIN S. GAME

SWAPPING

- A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution
  - Total physical memory space of processes can exceed physical memory
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk
- Does the swapped out process need to swap back in to same physical addresses?
- Depends on address binding method
  - Plus consider pending I/O to/from process memory space
- Context switch time can then be very high



Swapping of two processes using disk as a backing store

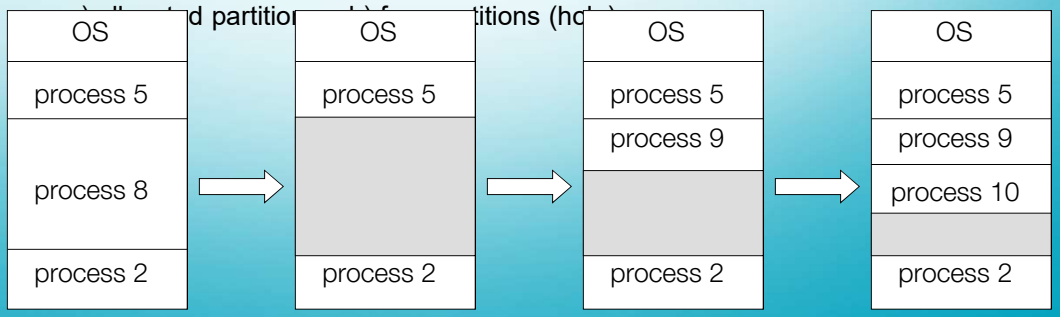
CONTIGUOUS ALLOCATION

- Main memory usually into two partitions:
  - Resident operating system, usually held in low memory with interrupt vector
  - User processes then held in high memory
  - Each process contained in single contiguous section of memory
- Relocation registers used to protect user processes from each other, and from changing operating-system code and data
  - Base register contains value of smallest physical address
  - Limit register contains range of logical addresses – each logical address must be less than the limit register
  - MMU maps logical address *dynamically*
  - Can then allow actions such as kernel code being **transient** and kernel changing size

PRAVIN S. GAME

Multiple-partition allocation

- Degree of multiprogramming limited by number of partitions
- Hole – block of available memory; holes of various size are scattered throughout memory
- When a process arrives, it is allocated memory from a hole large enough to accommodate it
- Process exiting frees its partition, adjacent free partitions combined
- Operating system maintains information about:



PRAVIN S. GAME

Dynamic Storage-Allocation Problem

- How to satisfy a request of size *n* from a list of free holes?
  - **First-fit**: Allocate the *first* hole that is big enough
  - **Best-fit**: Allocate the *smallest* hole that is big enough; must search entire list, unless ordered by size
    - Produces the smallest leftover hole
  - **Worst-fit**: Allocate the *largest* hole; must also search entire list
    - Produces the largest leftover hole
- First-fit and best-fit better than worst-fit in terms of speed and storage utilization
- Fragmentation
  - **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous. Results from first/best fit. Solution: **compaction.**
  - **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; unused memory that is internal to a partition.

PRAVIN S. GAME

PAGING

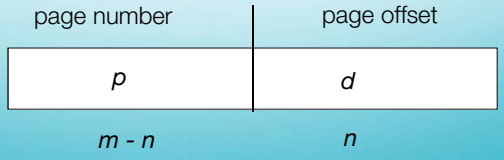
- MM scheme to allocate physical address space in non-contiguous manner.
- Avoids external fragmentation
- Backing store also split into pages

BASIC METHOD

- Divide physical memory into fixed-sized blocks called **frames**
  - Size is power of 2, between 512 bytes and 16 Mbytes
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size *N* pages, need to find *N* free frames and load program
- Set up a **page table** to translate logical to physical addresses

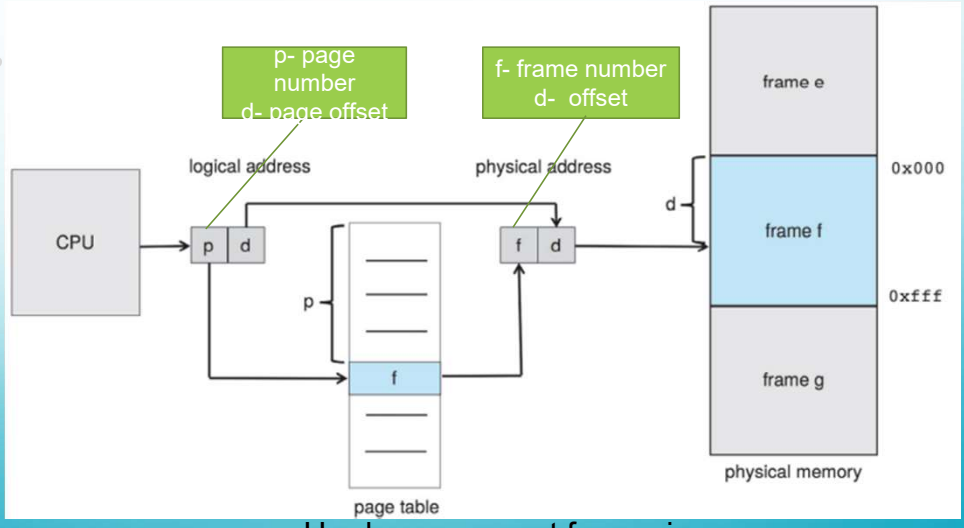
PRAVIN S. GAME

- Address generated by CPU is divided into:
  - **Page number (*p*)** – used as an index into a **page table** which contains base address of each page in physical memory
  - **Page offset (*d*)** – combined with base address to define the physical memory address that is sent to the memory unit
  - For given logical address space 2^m and page size 2^n

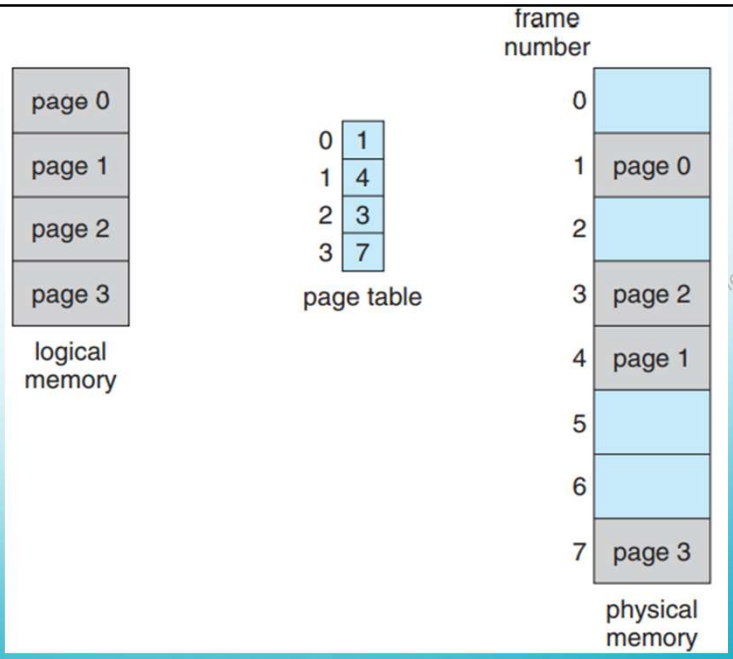


- Address translation

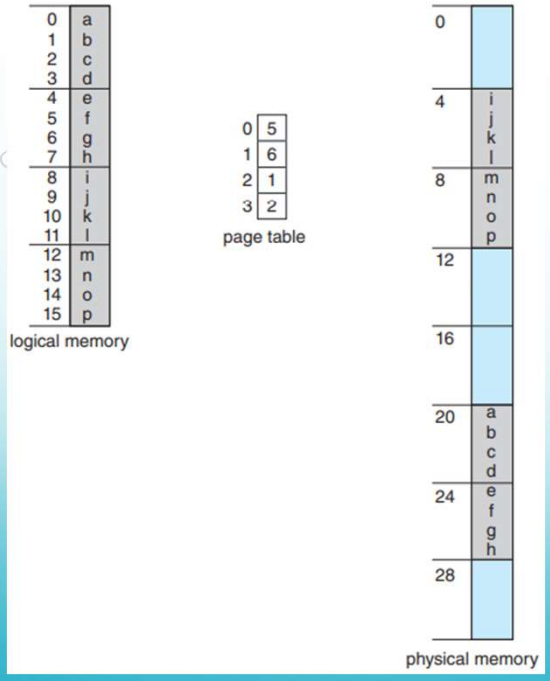
1. Extract the page number p and use it as an index into the page table.
2. Extract the corresponding frame number f from the page table.
3. Replace the page number p in the logical address with the frame number f



Hardware support for paging



Paging model of logical and physical memory



- With m = 4 and n = 2 i.e.

a 32-byte memory with 4-byte pages

- Logical address 3 (p= 0, d= 3)

maps to

Physical address 23 [(5 × 4) + 3]

- **Paging itself is a form of dynamic relocation**

PRAVIN S. GAME

- Problem of internal fragmentation

Let page size = 2 kB

Process size = 72766 bytes

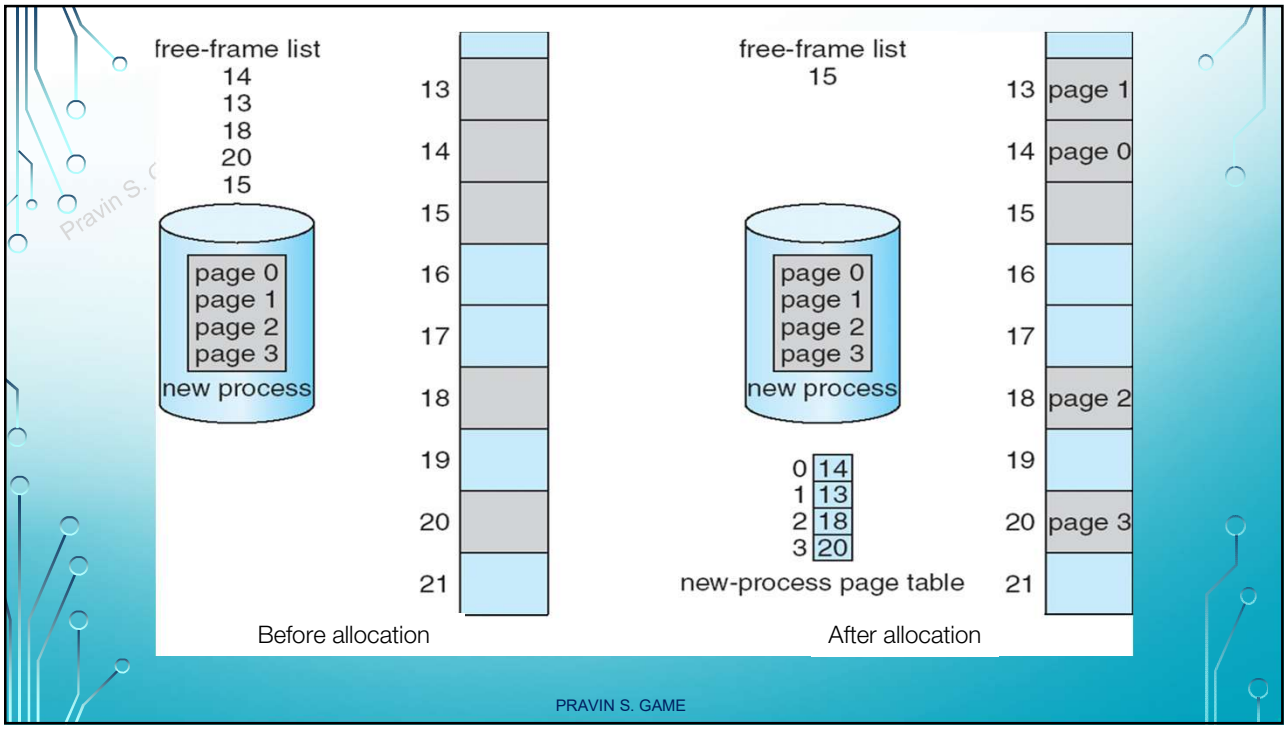
Pages required = 72766 / 2048 = 35.53 (35 pages + 1086 bytes)

System will allocate 36 pages…. i.e. 36 frames

Internal fragmentation 0f 2048-1086 = 962 bytes

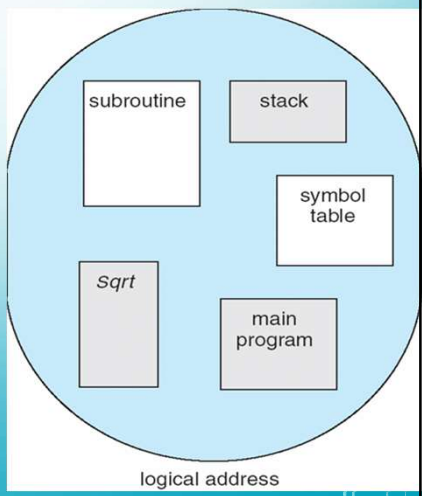
Worst case???

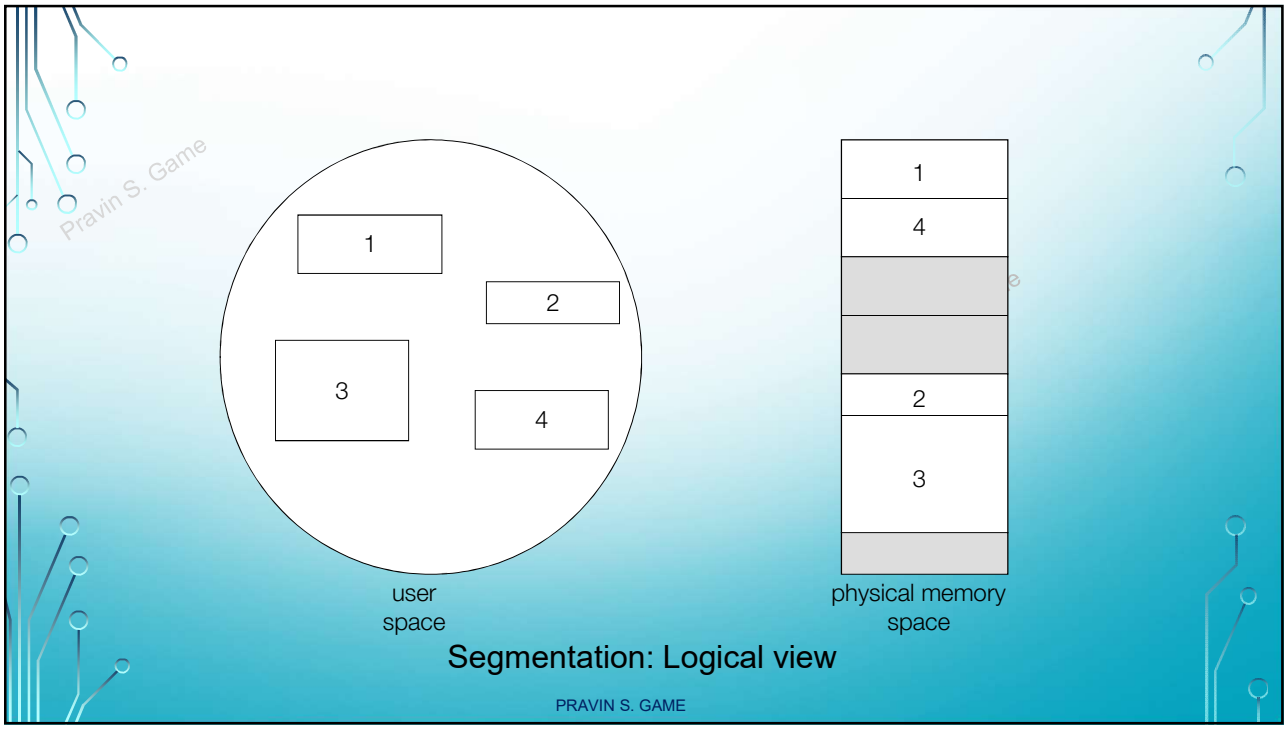
PRAVIN S. GAME



SEGMENTATION

- Memory-management scheme that supports user view of memory
- A program is a collection of segments
  - A segment is a logical unit such as:
    - main program
    - procedure
    - function
    - method
    - object
    - local variables, global variables
    - common block
    - stack
    - symbol table
    - arrays





Segmentation: Logical view

PRAVIN S. GAME

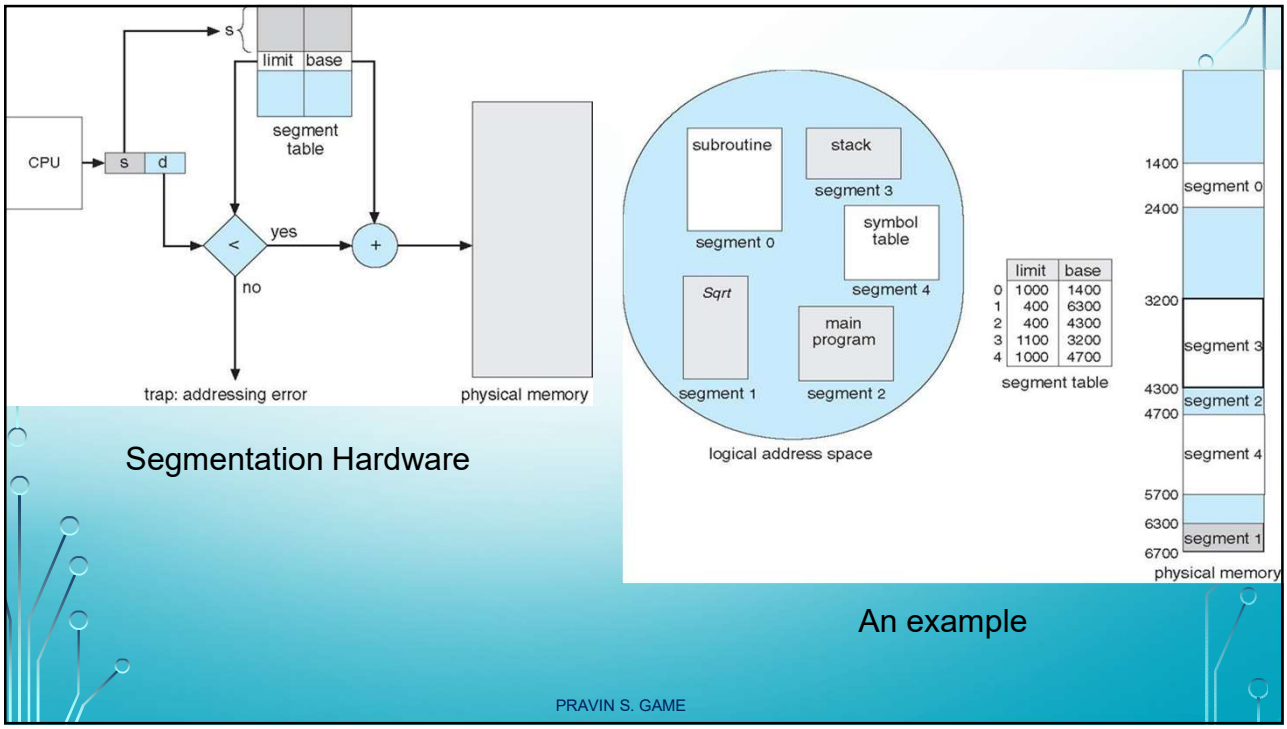
Segmentation Architecture

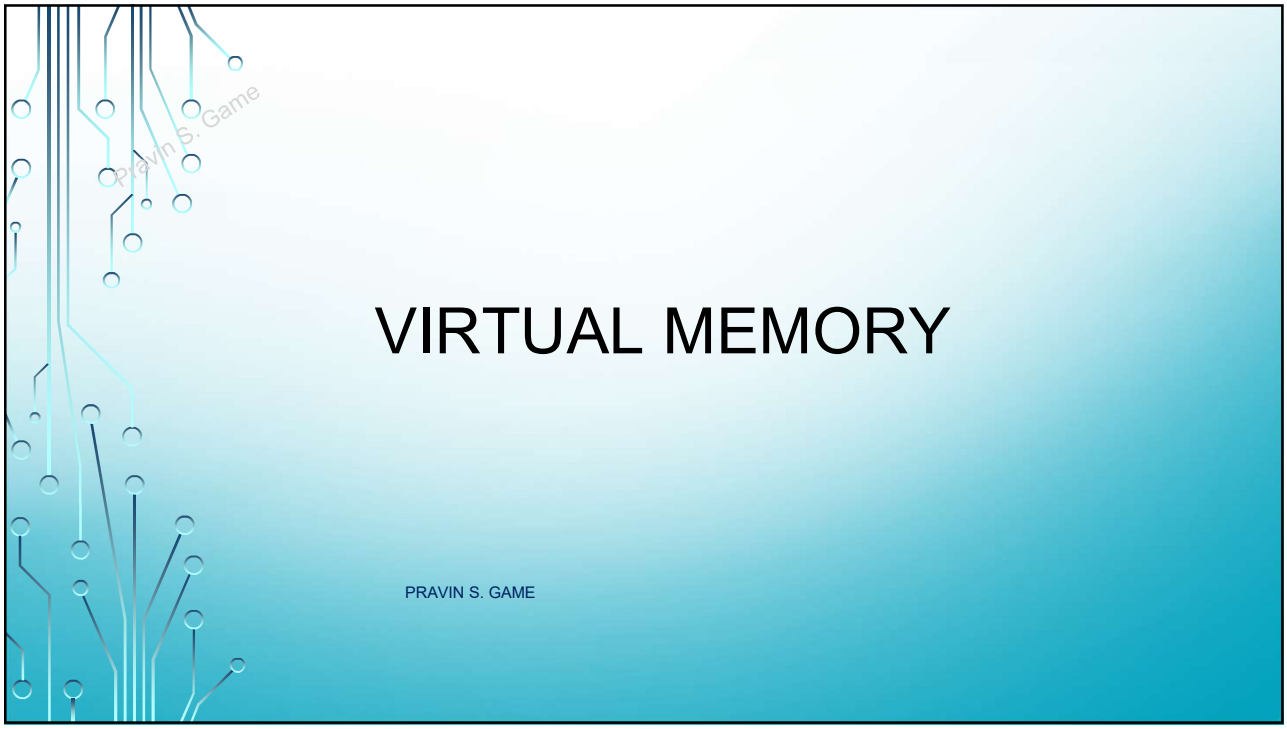
- Logical address consists of a two tuple: <segment-number, offset>,
- **Segment table** – maps two-dimensional physical addresses; each table entry has:
  - **base** – contains the starting physical address where the segments reside in memory
  - **limit** – specifies the length of the segment
- **Segment-table base register (STBR)** points to the segment table's location in memory
- **Segment-table length register (STLR)** indicates number of segments used by a program;

  segment number ***s*** is legal if ***s*** < **STLR**
- Protection

  With each entry in segment table associate:
  - validation bit = 0 ⇒ illegal segment
  - read/write/execute privileges

PRAVIN S. GAME

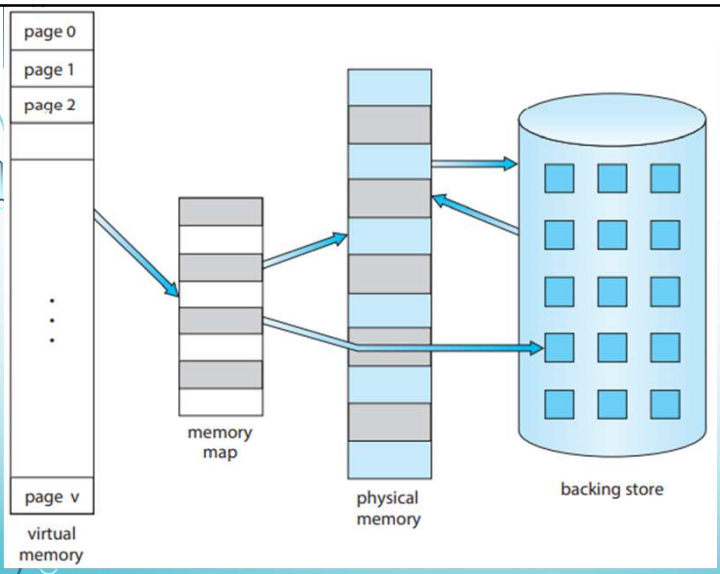

s
limit base
segment table
CPU
s d
<
yes
no
+
trap: addressing error
physical memory
Segmentation Hardware
subroutine
segment 0
stack
segment 3
symbol table
segment 4
Sqrt
segment 1
main program
segment 2
logical address space
limit base
0 1000 1400
1 400 6300
2 400 4300
3 1100 3200
4 1000 4700
segment table
1400
segment 0
2400
3200
segment 3
4300
segment 2
4700
segment 4
5700
6300
segment 1
6700
physical memory
An example
PRAVIN S. GAME


Pravin S. Game
VIRTUAL MEMORY
PRAVIN S. GAME

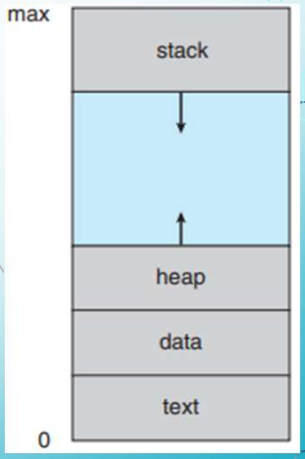
INTRODUCTION

- Entire logical address space in physical memory?
- Is this necessary?
- Exception routines, arrays/lists/tables more space, some functions rarely called.
- Entire program NOT needed all time
- Benefits of partially loaded program:
  - Program and programs could be larger than physical memory
  - Less I/O needed to load and swap
  - Benefits both user and system
- Virtual Memory:
  - Involves separation of logical memory as seen by users from physical memory
  - Provides extremely large virtual memory to programmers
  - Virtual address space refers to logical view of process in memory
  - Allows files and memory to be shared by processes : system libraries, shared regions/memory

PRAVIN S. GAME

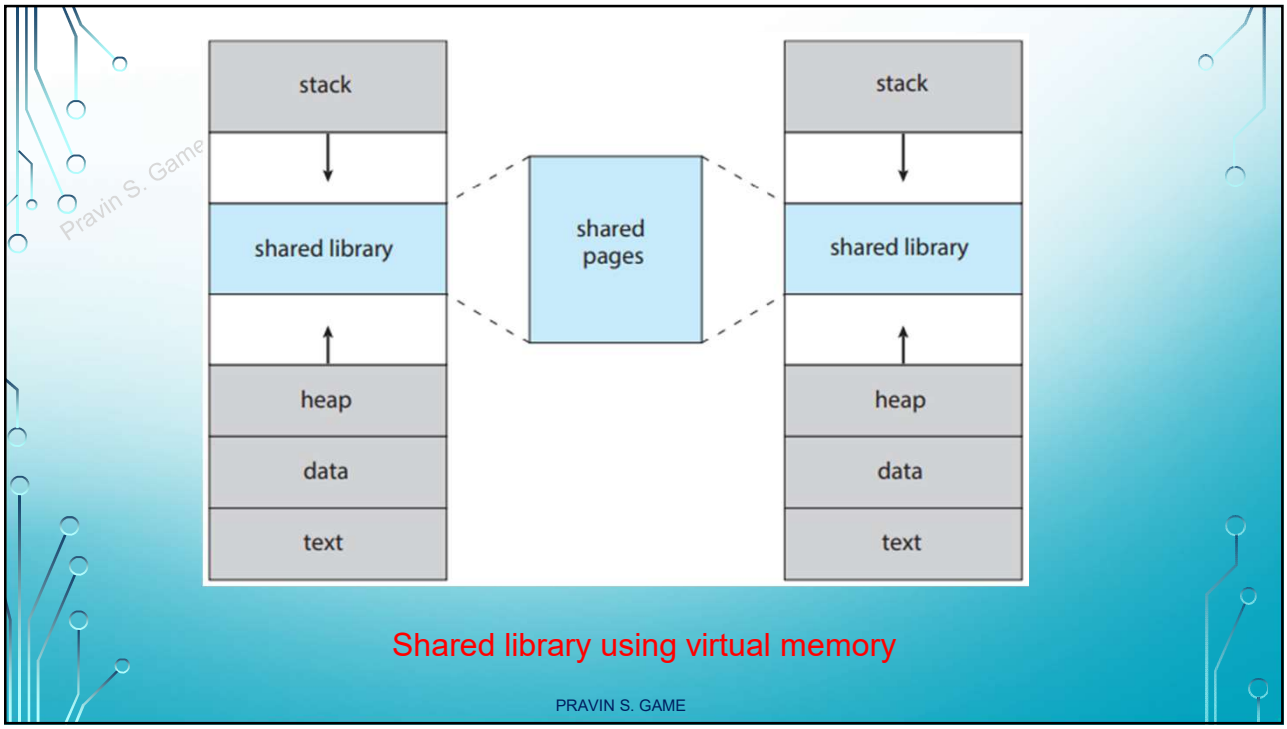


Virtual memory larger than physical memory



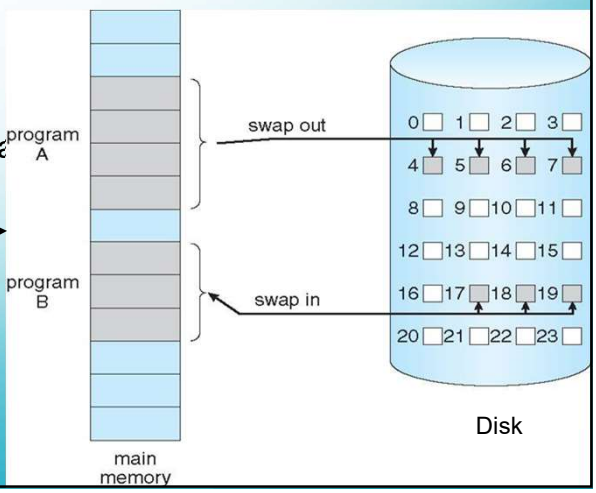
Virtual address space of process

PRAVIN S. GAME



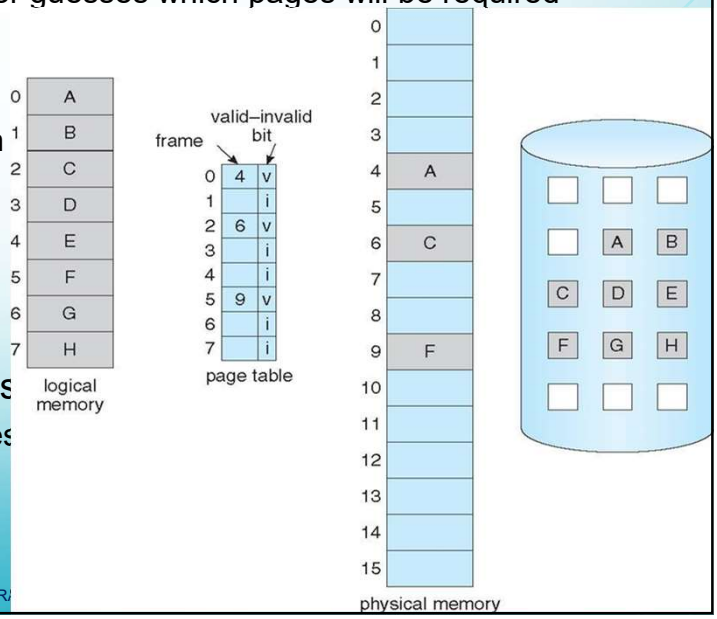
DEMAND PAGING

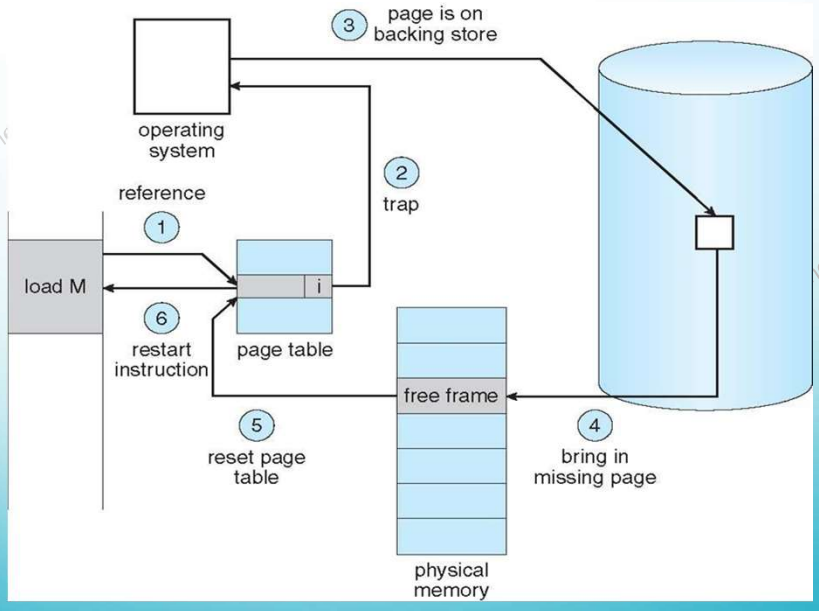
- Load entire program or partial program
- With partial program:
  - Less I/O and Memory needed
  - Improved response
  - More programs in memory
- Demand paging : load pages as they a demanded during execution
- Similar to paging with swapping
- Lazy swapper
- Instead of swapper call pager



PRAVIN S. GAME

- Process to be swapped in – pager guesses which pages will be required before swapping out
- So, loads only those pages
- Hardware support : to distinguish pages in memory and on disk
  - Modify page table : valid-invalid bit
- v – memory-resident & legal
- i – not in memory (but on dis
- Page fault: caused due to acces to page marked as invalid





Steps in handling a page fault

Pure Demand paging

- Start process with NO pages in memory
- OS sets instruction pointer to first instruction. It is on page which is not in memory
- Page fault occurs. Page is brought in memory.
- Execution continues…Till next page fault
- Once all required pages are in memory.. No more page faults

Requirements of demand paging:

- Hardware support : page table and secondary memory
- Ability to restart any instruction after a page fault.

- **Performance of Demand Paging**

1. Trap to the operating system.
2. Save the registers and process state.
3. Determine that the interrupt was a page fault.
4. Check that the page reference was legal, and determine the location of the page in secondary storage.
5. Issue a read from the storage to a free frame:
    a. Wait in a queue until the read request is serviced.
    b. Wait for the device seek and/or latency time.
    c. Begin the transfer of the page to a free frame.
6. While waiting, allocate the CPU core to some other process.

7. Receive an interrupt from the storage I/O subsystem (I/O completed).
8. Save the registers and process state for the other process (if step 6 is executed).
9. Determine that the interrupt was from the secondary storage device.
10. Correct the page table and other tables to show that the desired page is now in memory.
11. Wait for the CPU core to be allocated to this process again
12. Restore the registers, process state, and new page table, and then resume the interrupted instruction

- Let p be probability of page fault
- Page Fault Rate $0 \le p \le 1$
- Effective Access Time (EAT)

EAT = (1 – *p*) x memory access + *p* (page fault overhead + swap page out
+ swap page in + restart overhead)

Ex. If avg. page–fault service time = 8 milliseconds and
memory access time = 200 nanoseconds

EAT = (1-p) x 200 + p (8 millisec) = 200 + 7999800 p

- So EAT here is directly proportional to page-fault rate p.
- If p = 1/1000, EAT = 8299.8 millisec
- If expected performance degradation is less than 10%,

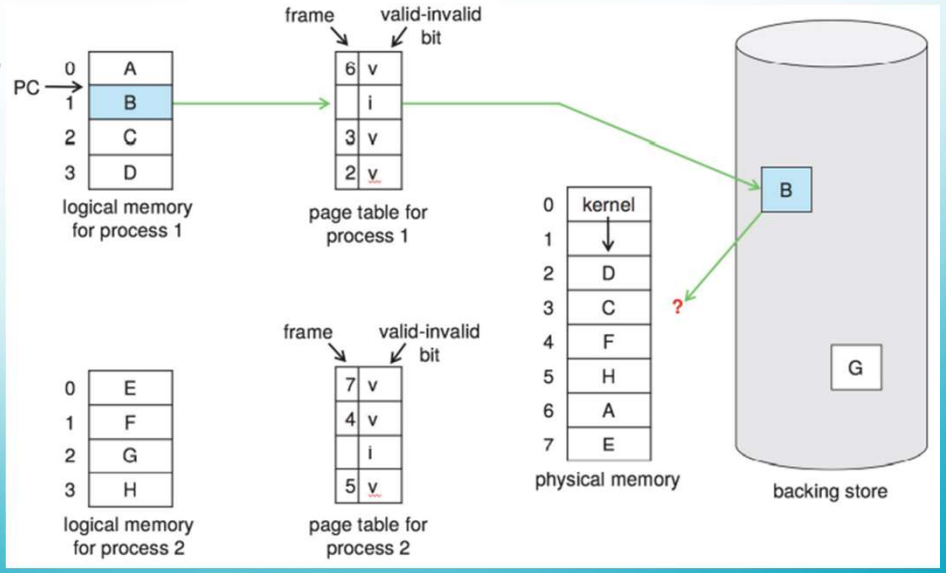
220 > 200 + 7999800 p
i.e. p < 0.0000025

one memory access out of 399990 can be allowed to cause page fault.

PAGE REPLACEMENT STRATEGIES

PRAVIN S. GAME

- Over-allocation of memory

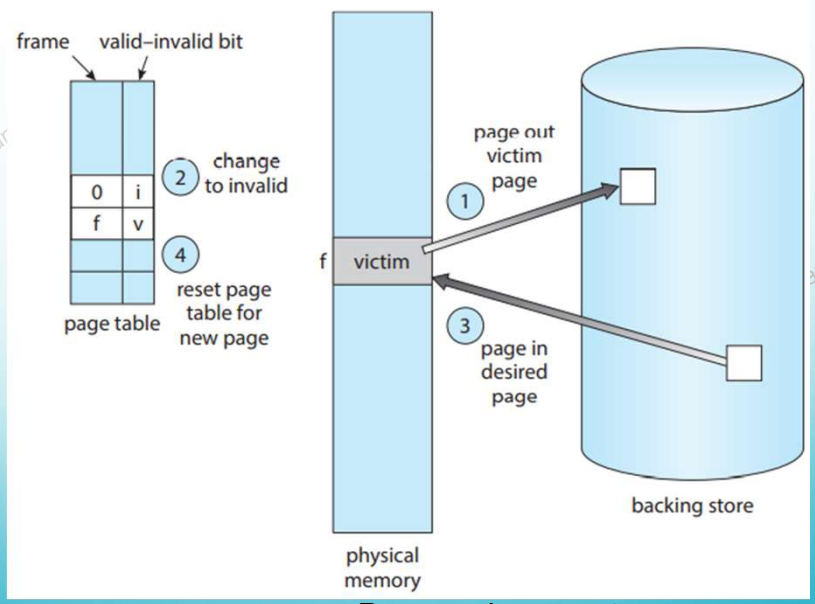


Need for page replacement

PRAVIN S. GAME

BASIC PAGE REPLACEMENT

1. Find the location of the desired page on secondary storage.
2. Find a free frame:
   a. If there is a free frame, use it.
   b. If there is no free frame, use a page-replacement algorithm to select a victim frame.
   c. Write the victim frame to secondary storage (if necessary); change the page and frame tables accordingly.
3. Read the desired page into the newly freed frame; change the page and frame tables.
4. Continue the process from where the page fault occurred.



Page replacement

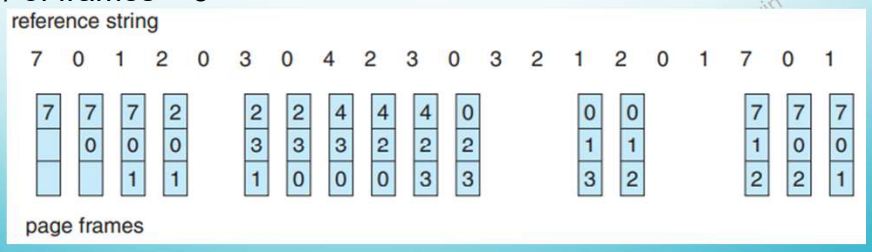
- Page replacement is basic to demand paging.
- It completes the separation between logical memory and physical memory.
- To implement demand paging, need :

1. **Frame allocation algorithm**
2. **Page replacement algorithm**
   - A. Basic
   - B. First-in, First-out
   - C. Optimal (replace the page that will not be used for the longest period of time)
   - D. Least recently used
   - E. Counting-based
   - F. Page-buffering algorithms

➢ Reference string
- string of memory references used to find page faults

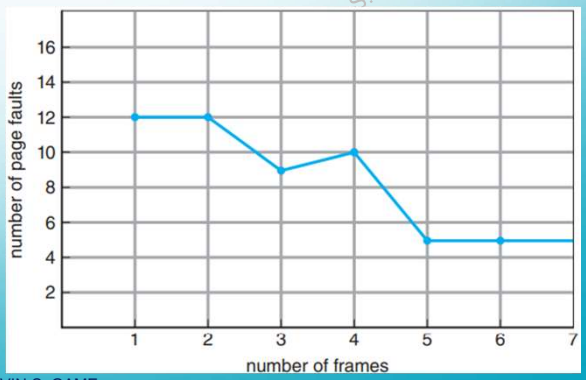
FIRST-IN, FIRST-OUT

- Associate time with page or maintain FIFO queue
- Replace oldest one or at the head of queue
- Reference string : 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
- # of frames = 3



- # of page faults = ?

- Easy to understand and code
- Performance is not always good.
- A bad replacement choice increases the page-fault rate and slows process execution.
- **Belady's anamoly**: for some page-replacement algorithms, the page-fault rate may increase as the number of allocated frames increases.
  - 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

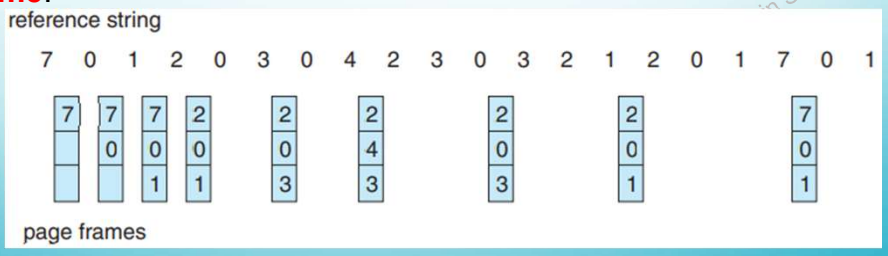


PRAVIN S. GAME

OPTIMAL PAGE REPLACEMENT

- Algorithm with lowest page-fault rate
- Does not suffer from Belady's anamoly
- Called as OPT or MIN

Replace the page that will not be used for the longest period of time.

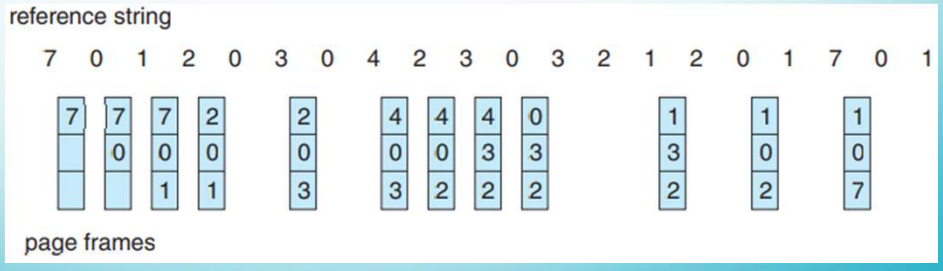


- # of page-faults = ?
- Difficult to implement

PRAVIN S. GAME

LRU PAGE REPLACEMENT

- replace the page that has not been used for the longest period of time.
- Associates time of that page's last use
- OPT looking backward in time???



- # of page-faults?
- Good. Often used. Needs h/w assistance.
- Two ways to implement: counter and stack

- Counter implementation
  - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
  - When a page needs to be changed, look at the counters to find smallest value
    - Search through table needed
- Stack implementation
  - Keep a stack of page numbers in a double link form:
  - Page referenced:
    - move it to the top
    - requires 6 pointers to be changed
  - But each update more expensive
  - No search for replacement
- LRU and OPT are cases of **stack algorithms** that don't have Belady's Anomaly

Apply the (1) FIFO, (2) LRU, and (3) optimal (OPT) replacement algorithms for the following page-reference strings:

- 2, 6, 9, 2, 4, 2, 1, 7, 3, 0, 5, 2, 1, 2, 9, 5, 7, 3, 8, 5
- 0, 6, 3, 0, 2, 6, 3, 5, 2, 4, 1, 3, 0, 6, 1, 4, 2, 3, 5, 7
- 3, 1, 4, 2, 5, 4, 1, 3, 5, 2, 0, 1, 1, 0, 2, 3, 4, 5, 0, 1
- 4, 2, 1, 7, 9, 8, 3, 5, 2, 6, 8, 1, 0, 7, 2, 4, 1, 3, 5, 8
- 0, 1, 2, 3, 4, 4, 3, 2, 1, 0, 0, 1, 2, 3, 4, 4, 3, 2, 1, 0

Indicate the number of page faults for each algorithm assuming demand paging with three frames.

REFERENCES

- Silberschatz, Galvin, Gagne, "Operating System Principles", 10th Edition
- Stallings W., "Operating Systems- internals and design principles", 9th Edition.
- Deitel, Deitel, Choffnes, “Operating System”, 3rd edition, Pearson